

Declaración de uso de IA Entrega 2

Vicente Vial:

IA usada: ChatGPT.

Alcance y contexto:

En mi parte de la Entrega 2 usé ChatGPT para entender e implementar las vistas de inicio de sesión y registro de usuarios en React.

La IA fue usada para responder dudas sobre la estructura de los componentes, el uso de formularios, inputs, botones, manejo de eventos y estados con useState, también la usé para entender cómo separar la lógica del componente en JSX de los estilos en CSS y para revisar propiedades necesarias para construir estas vistas, como Flexbox, imágenes de fondo, espaciado y estilos de botones.

Además pregunté cómo simular el registro e inicio de sesión mientras el proyecto sigue trabajando con datos mock y todavía no cuenta con una conexión al backend, también revisé cómo cargar usuarios desde un archivo JSON y cómo agregar temporalmente nuevos usuarios utilizando el estado de React.

La IA fue utilizada principalmente como herramienta de apoyo y referencia.

Aporte y Análisis Crítico:

Usé a Chatgpt como apoyo para entender cómo estructurar las vistas y manejar formularios en React. A partir de esos ejemplos adapté el código a la estructura y necesidades específicas del proyecto.

En la Landing Page tomé ideas generales de distribución, botones y estilos, pero se ajustaron al diseño del juego, agregué el logo, la imagen de fondo, la tipografía utilizada en el proyecto y la navegación hacia Login, Register y Tutorial con React Router. También modifiqué los tamaños, espacios y distribución para que la página se adapte mejor a distintos tamaños de pantalla.

En Login, los ejemplos iniciales con useState y formularios los adapté para trabajar con los usuarios mock del proyecto también agregué validaciones de campos vacíos, revisión de usuario y contraseña, mensajes de error y navegación al menú cuando los datos son correctos.

En Register se amplió bastante la estructura inicial porque agregué campos como confirmación de contraseña, tipo de usuario y código de administrador, además de validaciones para evitar usuarios o correos repetidos y mostrar errores según cada caso.

Como en esta entrega todavía no existe un backend conectado, el registro de nuevos usuarios lo simulo usando los datos mock. Esto permite probar el flujo de registro e inicio de sesión pero es una solución temporal, ya que los datos no se guardan de forma permanente y las contraseñas no tienen un manejo seguro como tendría una aplicación real.

En general, la IA me sirvió como referencia para entender conceptos y obtener ejemplos iniciales, pero revisé el código, lo modifiqué e integré para ajustarlo al diseño, la navegación y los datos usados en el proyecto.

Tomás Mancilla:

IA usada: Claude

Alcance y contexto:

En mi parte de la Entrega 2 utilicé Claude como apoyo para implementar responsividad en todos los aspectos del proyecto, ayudándome a entender cómo usar clamp y @media en todo el proyecto.

También la utilicé para revisar, completar y estructurar el borrador del protocolo JSON de la Entrega 1. En particular, consulté sobre los casos en que corresponde utilizar peticiones REST o WebSocket, y cómo estructurar las peticiones para que no haya redundancias o información valiosa que no se esté enviando en los mensajes. Por otro lado, consulte la IA para entender cómo implementar un navbar que cambia según el usuario que inició sesión y las páginas que visita.

Además, consulté dudas de configuración del archivo esling.config.mjs (un error impedía reconocer variables globales del navegador).

Aporte y Análisis Crítico:

Claude fue utilizada principalmente como una herramienta de apoyo para comprender conceptos, revisar decisiones y detectar posibles inconsistencias. En el protocolo de comunicación, sus sugerencias me permitieron identificar elementos que debían estar presentes y aclarar cuándo correspondía utilizar REST o WebSocket. Las modificaciones propuestas fueron evaluadas y aplicadas directamente por mí.

En la implementación del navbar, la IA me ayudó a comprender cómo adaptar los elementos visibles según el usuario que inició sesión y la página en la que se encuentra. Utilicé estas explicaciones como guía para implementar la solución dentro de la estructura existente del proyecto y posteriormente verifiqué su funcionamiento.

En el caso de ESLint, el problema fue identificado por mí y utilicé la IA para comprender su causa y encontrar una posible solución. Posteriormente, verifiqué que la configuración propuesta resolviera efectivamente el error.

En general, la IA cumplió un rol de apoyo durante el desarrollo, principalmente para explicar conceptos, revisar lo realizado y orientar la resolución de problemas. Las decisiones de diseño, la evaluación de las sugerencias, la verificación de los cambios y la integración final al proyecto fueron realizadas por nosotros.

Miguel Mujica

IA usada: Claude Code

Alcance y contexto:

Utilice la IA para darle acceso a la carpeta y trabajar en conjunto para la redacción de código y creación de archivos, en donde acorde a mis tareas fuimos punto a punto haciendo lo que buscaba basándonos tanto en la entrega 1 como en ideas propias.

La IA fue la responsable del código y yo del análisis de este, ya sea marcando objetivos y revisando todo lo entregado con una mirada crítica tanto para retroalimentar o aceptar lo recibido, haciendo en ciertos momentos dudas a la IA sobre el código para comprender lo que se hacía.

Aporte y Análisis Crítico:

El tablero quedó indexado con Maps por clave "x,y" a partir de las casillas visibles y de los contactos que llegan del mock, así la niebla no se calcula acá sino que es simplemente una casilla que no viene en el mapa. Esto importa porque el servidor filtra el estado antes de mandarlo y nunca entrega la flota enemiga completa, por lo que si el cliente calculara la visibilidad bastaría con abrir la consola para hacer trampa. Por la misma razón BoardCell quedó sin nada de lógica de juego, recibe flags por props y solo pinta, y es un button con disabled cuando no hay onClick, para que se pueda navegar con el teclado. En el panel las 3 acciones del turno son items que dicen "Disponible" o "Usada" y no solo color, y el panel no guarda estado propio, ya que los valores son del turno y no del panel. En el cambio de turno la ronda sube solo cuando el turno vuelve a mí, porque si subiera cada vez que cambia el jugador la partida terminaría a los 20 turnos en vez de las 20 rondas completas; recién en ese momento se devuelven las 3 acciones, se limpia el contador de acciones por

barco y se saca una carta de evento nueva, y durante el turno del rival no se reinicia nada. Asumí a propósito que el turno del rival es un botón de simulación, ya que todavía no hay backend ni WebSocket, y que la niebla no se recalcula al mover un barco. En Preparación, el colocar al azar recalcula las casillas libres en cada iteración para que dos barcos no puedan caer en la misma casilla; hacer click sobre un barco ya puesto lo selecciona en vez de sobrescribirlo, el foco avanza solo al siguiente barco sin desplegar, y al llegar el reloj de 120 segundos a 0 se completa al azar y se navega igual, imitando lo que va a hacer el servidor. Todo esto está en `src/components/Tablero/`, `src/components/BoardCell/`, `src/components/PanelRecursos/`, `src/pages/Preparacion/` y en la función `terminarTurno` de `src/pages/Partida/Partida.jsx`.

Capturas de pantalla

explicame las bases para implementar una página de inicio de sesión en una pagina web

Para implementar una página de inicio de sesión, conviene entender primero el flujo completo. Visualmente es simple, pero detrás hay varias piezas.

1. El formulario de login

Como mínimo necesitas dos campos:

```
</> txt
Correo
Contraseña

[ Iniciar sesión ]
```

En HTML sería algo conceptualmente así:

```
</> HTML
<form>
  <input type="email" placeholder="Correo">
  <input type="password" placeholder="Contraseña">
  <button type="submit">Iniciar sesión</button>
</form>
```

La idea importante es que ambos inputs pertenecen a un `<form>`, porque quieres ejecutar una acción cuando el usuario presione el botón o Enter.

2. Guardar lo que escribe el usuario

Si estás usando React, normalmente tienes un estado para cada input:

```
</> JavaScript
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
```

Y conectas los inputs:

```
</> JavaScript
<input
  type="email"
  value={email}
  onChange={e => setEmail(e.target.value)}
/>
```

Entonces:

```
</> txt
Usuario escribe
↓
onChange
```

Entonces:

```
</> txt
Usuario escribe
↓
onChange
↓
setEmail(...)
↓
email contiene lo escrito
```

Con contraseña haces exactamente lo mismo.

3. Capturar el envío del formulario

Cuando el usuario aprieta **Iniciar sesión**, quieres ejecutar una función.

Por ejemplo:

```
</> JavaScript
const handleSubmit = (e) => {
  e.preventDefault();

  console.log(email);
  console.log(password);
};
```

Y:

```
</> JavaScript
<form onSubmit={handleSubmit}>
```

Ej:

```
</> JavaScript
e.preventDefault();
```

evita que el navegador recargue la página al enviar el formulario.

Esta parte es fundamental.

4. Validar los datos

Antes de intentar iniciar sesión, revisas cosas básicas.

Por ejemplo:

```
</> JavaScript
if (email === "" || password === "") {
  alert("Completa todos los campos");
  return;
}
```

+ Preguntar a ChatGPT

Alta

🗲️ 🔊

Por ejemplo:

```
JavaScript
if (email === "" || password === "") {
  alert("Completa todos los campos");
  return;
}
```

También puedes revisar:

```
txt
¿El correo tiene formato válido?
¿la contraseña está vacía?
¿tiene la longitud mínima?
```

Pero hay una distinción importante:

Frontend:

```
txt
"¿El usuario escribió algo razonable?"
```

Backend:

```
txt
"¿Este usuario realmente existe?"
"¿la contraseña es correcta?"
```

El frontend no debería decidir si las credenciales son correctas.

5. Enviar los datos al backend

En una aplicación real, el frontend manda las credenciales al servidor.

Por ejemplo:

```
JavaScript
fetch("http://localhost:3000/login", {
  method: "POST",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify({
    email: email,
    password: password
  })
});
```

Estarías enviando algo parecido a:

```
JSON
{
  "email": "usuario@mail.com",
  "password": "123456"
}
```

+ Preguntar a ChatGPT

Estarías enviando algo parecido a:

```
JSON
{
  "email": "usuario@mail.com",
  "password": "123456"
}
```

Al endpoint:

```
txt
POST /login
```

6. El backend revisa las credenciales

El backend recibe:

```
txt
correo
contraseña
```

Y busca al usuario en la base de datos:

```
txt
usuarios
-----
id
email
password_hash
```

Conceptualmente:

```
txt
Frontend
email + password
  ↓
POST /login
  ↓
Backend
  ├── busca usuario
  └── verifica contraseña
  ↓
¿Correcto?
```

```
txt
  ↓
POST /login
  ↓
Backend
  ├── busca usuario
  └── verifica contraseña
  ↓
{Correcto}
  |
  +--> ok
```

7. Nunca guardar contraseñas directamente

Esto es importantísimo.

No deberías tener una base de datos así:

```
txt
email      password
juan@mail.com  hola123
```

Se guarda una versión transformada mediante hash:

```
txt
email      password_hash
juan@mail.com  $2b$12$4zjf...
```

Normalmente se utilizan algoritmos/librerías como:

```
txt
bcrypt
Argon2
```

Así el servidor compara la contraseña introducida contra el hash.

8. Mantener al usuario conectado

Una vez que el servidor confirma:

```
txt
correo correcto
+
contraseña correcta
```

necesitas alguna manera de recordar:

+ Preguntar a ChatGPT

```
txt
correo correcto
+
contraseña correcta
```

necesitas alguna manera de recordar:

"Este usuario ya inició sesión."

Normalmente se hace con:

Sesiones + cookies

o

Tokens

Por ejemplo:

```
txt
Login correcto
+
Servidor crea sesión
+
Navegador recibe cookie
+
Usuario entra a /menu
```

Después, cuando el usuario pide algo privado:

```
txt
GET /perfil
```

el navegador manda la cookie y el servidor sabe quién es.

Para un proyecto universitario probablemente esto lo manejará el backend que tengan implementado.

9. Redireccionar después del login

Si el inicio de sesión funciona:

```
txt
/login
+
usuario correcto
+
/menu
```

En React Router normalmente terminarías haciendo algo como:

```
JavaScript
navigate("/menu");
```

Si falla:

JavaScript

navigate("/menu");

+

🔗

Si falla:

txt

login

↓

contraseña incorrecta

↓

"Correo o contraseña incorrectos"

+

🔗

10. Manejar errores visualmente

En vez de solamente usar `!alert()`, normalmente tienes otro estado:

JavaScript

const [error, setError] = useState("");

+

🔗

Por ejemplo:

JavaScript

if (loginIncorrecto) {
 setError("Correo o contraseña incorrectos");
}

+

🔗

Y en el HTML:

JavaScript

{error && <p>{error}</p>}

+

🔗

Entonces aparece debajo del formulario.

La estructura básica completa

Mentalmente puedes pensar el componente así:

JavaScript

function Login() {

 // 1. Estados
 const [email, setEmail] = useState("");
 const [password, setPassword] = useState("");
 const [error, setError] = useState("");

 // 2. Función de login
 const handleSubmit = (e) => {

 e.preventDefault();

 // 3. Validar inputs

 // 4. Enviar email/password al backend

 // 5. Revisar respuesta

 // 6. Si funciona + ir al menú

 // 7. Si falla + mostrar error

 };
}

+

🔗

+

Preguntar a ChatGPT

Alta

🔍

🔊

La estructura básica completa

Mentalmente puedes pensar el componente así:

JavaScript

function Login() {

 // 1. Estados
 const [email, setEmail] = useState("");
 const [password, setPassword] = useState("");
 const [error, setError] = useState("");

 // 2. Función de login
 const handleSubmit = (e) => {

 e.preventDefault();

 // 3. Validar inputs

 // 4. Enviar email/password al backend

 // 5. Revisar respuesta

 // 6. Si funciona + ir al menú

 // 7. Si falla + mostrar error

 };
}

+

🔗

Eso es prácticamente el esqueleto de cualquier página de inicio de sesión en React.

Para tu proyecto web, lo más importante es separar mentalmente estas tres cosas:

txt

INTERFAZ
Inputs, botón, estilos
↓

LÓGICA FRONTEND
useState
handleSubmit
validaciones
↓

BACKEND
buscar usuario
verificar contraseña
crear sesión/token

+

🔗

Y si todavía están haciendo solo frontend, puedes implementar perfectamente las primeras dos partes y simular temporalmente que el login funciona, por ejemplo haciendo que `handleSubmit` te mande a `/menu`. Después, cuando esté listo el backend, reemplazaras esa simulación por la petición real.

y con react que habría que cambiar?

Con React, la idea general **no cambia**, lo que cambia es cómo manejas la interfaz y los datos. En vez de leer directamente los inputs desde el HTML, normalmente usas **estado (useState)**.

Lo básico sería esto:

```
JavaScript
import { useState } from "react";

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();

    console.log("Email:", email);
    console.log("Password:", password);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        placeholder="Correo"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
    </form>
  );
}
```

La diferencia importante está acá:

```
JavaScript
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");

En React, el componente va guardando lo que escribe el usuario.
Por ejemplo:
```

```
JavaScript
onChange={(e) => setEmail(e.target.value)}
```

Cada vez que escribes una letra:

```
txt
usuario escribe
↓
onChange
↓
setEmail(...)
↓
React actualiza email
```

Preguntar a ChatGPT

Alta

```
JavaScript
onChange={(e) => setEmail(e.target.value)}
```

Cada vez que escribes una letra:

```
txt
usuario escribe
↓
onChange
↓
setEmail(...)
↓
React actualiza email
```

Entonces ya no necesitas hacer algo típico de JavaScript puro como:

```
JavaScript
document.getElementById("email").value
```

En React eso normalmente se evita.

También el envío cambia un poco. En HTML tradicional podrías hacer:

```
HTML
<form action="/login" method="POST">
```

En React normalmente haces:

```
JavaScript
<form onSubmit={handleSubmit}>
```

Y manejas tú la lógica:

```
JavaScript
const handleSubmit = (e) => {
  e.preventDefault();

  // validar
  // enviar al backend
  // mostrar error o redirigir
};
```

Si tienes backend, dentro de `handleSubmit` podrías hacer:

```
JavaScript
const handleSubmit = async (e) => {
  e.preventDefault();

  const response = await fetch("http://localhost:3000/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      email,
      password,
    }),
  });

  const data = await response.json();
  console.log(data);
};
```

Preguntar a ChatGPT

Alta

```
JavaScript
const handleSubmit = async (e) => {
  e.preventDefault();

  const response = await fetch("http://localhost:3000/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      email,
      password,
    }),
  });

  const data = await response.json();
  console.log(data);
};
```

Fíjate además que esto:

```
JavaScript
{
  email,
  password
}
```

es simplemente una forma corta de escribir:

```
JavaScript
{
  email: email,
  password: password
}
```

Y si estás usando **React Router**, puedes redirigir al usuario cuando el login sea correcto:

```
JavaScript
import { useNavigate } from "react-router-dom";

y dentro del componente:
```

```
JavaScript
const navigate = useNavigate();

Luego:
```

```
JavaScript
if (loginCorrecto) {
  navigate("/menu");
}
```

Así que, para resumir, en React principalmente agregas:

Preguntar a ChatGPT

Alta

Así que, para resumir, en React principalmente agregas:

```
txt
useState
↓
guardar email/password

onChange
↓
actualizar esos estados

onSubmit
↓
ejecutar handleSubmit

fetch
↓
hablar con backend

useNavigate
↓
cambiar de página
```

Para tu proyecto, una estructura típica sería algo así:

```
txt
src/
├── pages/
│   ├── Login.jsx
│   ├── Register.jsx
│   ├── Landing.jsx
│   └── Menu.jsx
├── components/
│   └── Navbar.jsx
└── App.jsx
```

Y `Login.jsx` contiene principalmente el formulario y la lógica del login.

Si todavía no tienen backend conectado, puedes hacer una versión mucho más simple donde el botón solo valide que haya correo/contraseña y luego haga:

```
JavaScript
navigate("/menu");
```

Eso te permite dejar **toda la interfaz funcionando primero** y conectar el backend después.

Preguntar a ChatGPT

Alta

eso de ahí es solo el jsx o no? cuales son las bases para el css?

Si, lo de antes era básicamente el JSX + lógica de React. El CSS va aparte y se encarga solo de cómo se ve.

Por ejemplo, podrías tener:

```
</> txt
src/
├── pages/
│   ├── login.jsx
│   └── login.css
```

En login.jsx importas el CSS:

```
</> JavaScript
import './login.css';
```

Y en el JSX le pones clases a los elementos:

```
</> JavaScript
function login() {
  return (
    <div className="login-page">
      <form className="login-form">
        <h1>Iniciar sesión</h1>
        <input
          className="login-input"
          type="email"
          placeholder="Correo"
        />
        <input
          className="login-input"
          type="password"
          placeholder="Contraseña"
        />
        <button className="login-button">
          Iniciar sesión
        </button>
      </form>
    </div>
  );
}
```

La idea es:

```
</> txt
JSX
├── define qué elementos existen
└── CSS
    ├── define cómo se ven esos elementos
```

+ Preguntar a ChatGPT

Alta

```
</> txt
├── define qué elementos existen
└── CSS
    ├── define cómo se ven esos elementos
```

Por ejemplo, las bases del CSS para una página de login serían estas.

```
</> CSS
.login-page {
  min-height: 100vh;

  display: flex;
  justify-content: center;
  align-items: center;

  background-color: #f5f5f5;
}
```

Esto hace que todo el login ocupe la pantalla completa y quede centrado.

Lo importante acá es:

```
</> CSS
display: flex;
justify-content: center;
align-items: center;
```

Con eso centras el formulario horizontal y verticalmente.

Luego el formulario:

```
</> CSS
.login-form {
  width: 350px;

  display: flex;
  flex-direction: column;
  gap: 15px;

  padding: 30px;

  background-color: white;
  border-radius: 10px;
}
```

Aquí hay otra cosa importante:

```
</> CSS
flex-direction: column;
```

Eso hace que los elementos queden uno debajo del otro:

```
</> txt
├──
└──
```

+ Preguntar a ChatGPT

Alta

```
</> CSS
flex-direction: column;
```

Eso hace que los elementos queden uno debajo del otro:

```
</> txt
Iniciar sesión

[ correo ]

[ contraseña ]

[ botón ]
```

Y:

```
</> CSS
gap: 15px;
```

deja espacio entre ellos.

Para los inputs:

```
</> CSS
.login-input {
  padding: 12px;

  border: 1px solid #ccc;
  border-radius: 5px;

  font-size: 16px;
}
```

Y cuando el usuario hace clic en un input puedes cambiar cómo se ve:

```
</> CSS
.login-input:focus {
  outline: none;
  border-color: #0066ff;
}
```

Ese:

```
</> CSS
:focus
```

significa:

aplica este estilo cuando el usuario está escribiendo en ese input.

Para el botón:

```
</> CSS
```

+ Preguntar a ChatGPT

Alta

significa:

aplica este estilo cuando el usuario está escribiendo en ese input.

Para el botón:

```
</> CSS
.login-button {
  padding: 12px;

  border: none;
  border-radius: 5px;

  background-color: #0066ff;
  color: white;

  font-size: 16px;
  cursor: pointer;
}
```

Y normalmente haces que cambie al pasar el mouse:

```
</> CSS
.login-button:hover {
  background-color: #0052cc;
}
```

Eso es el hover.

Entonces un CSS básico completo podría ser:

```
</> CSS
.login-page {
  min-height: 100vh;

  display: flex;
  justify-content: center;
  align-items: center;

  background-color: #f5f5f5;
}

.login-form {
  width: 350px;

  display: flex;
  flex-direction: column;
  gap: 15px;

  padding: 30px;

  background-color: white;
  border-radius: 10px;
}
```

Las cosas que deberías manejar para hacer prácticamente cualquier login son:

```
</> txt
1. width / height
   tamaño de los elementos
```

+ Preguntar a ChatGPT

Alta

```
2. margin / padding
   espacios

3. display: flex
   organizar elementos

4. flex-direction
   fila o columna

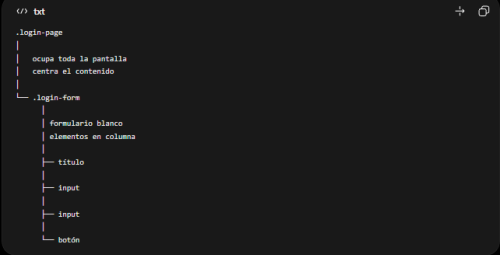
5. justify-content
   posición horizontal

6. align-items
   posición vertical

7. gap
   separación entre elementos

8. background-image
```

Y para un login, la estructura mental típica es:



Una cosa especialmente importante en React: en HTML normal usarías:

```
</> HTML
<div class="login">
```

pero en React escribes:

```
</> JavaScript
<div className="login">
```

Porque `class` no se usa directamente en JSX.

Si estás aprendiendo esto para poder implementarlo tú solo, yo me enfocaría primero en dominar `flex`, `padding`, `margin`, `width`, `gap` y clases CSS. Con esas seis cosas ya puedes construir prácticamente toda la estructura visual de un login sencillo.

Si estás aprendiendo esto para poder implementarlo tú solo, yo me enfocaría primero en dominar `flex`, `padding`, `margin`, `width`, `gap` y clases CSS. Con esas seis cosas ya puedes construir prácticamente toda la estructura visual de un login sencillo.

y como se puede poner una foto de fondo en el CSS?

Sí. En CSS se hace con `background-image`.

Por ejemplo, si tienes una imagen llamada `fondo.jpg`:

```
</> CSS
.login-page {
  min-height: 100vh;

  background-image: url("/fondo.jpg");
}
```

Pero normalmente también agregas estas propiedades:

```
</> CSS
.login-page {
  min-height: 100vh;

  background-image: url("/fondo.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;

  display: flex;
  justify-content: center;
  align-items: center;
}
```

Lo importante es:

```
</> CSS
background-size: cover;
```

Hace que la imagen cubra toda la pantalla, aunque tenga que recortarse un poco.

```
</> CSS
background-position: center;
```

Mantiene el centro de la imagen visible.

```
</> CSS
background-repeat: no-repeat;
```

Evita que la imagen se repita.

En React, la ruta depende de dónde guardes la imagen.

Si está en `public`:



USAS:

```
</> CSS
background-image: url("/fondo.jpg");
```

Si está dentro de `src`, por ejemplo:



Y `login.css` está dentro de `pages`, normalmente sería:

```
</> CSS
background-image: url("../assets/fondo.jpg");
```

También puedes poner una capa oscura encima, que para logins queda bastante útil porque hace que el formulario resalte más:

```
</> CSS
.login-page {
  min-height: 100vh;

  background-image:
    linear-gradient(
      rgba(0, 0, 0, 0.4),
      rgba(0, 0, 0, 0.4)
    ),
    url("/fondo.jpg");

  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;

  display: flex;
  justify-content: center;
  align-items: center;
}
```

Ese `linear-gradient`: negro con transparencia oscurece la foto sin modificar la imagen original.

Ese `linear-gradient`: negro con transparencia oscurece la foto sin modificar la imagen original.

Para tu login, la versión típica sería:

```
</> CSS
.login-page {
  min-height: 100vh;

  background-image: url("/fondo.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;

  display: flex;
  justify-content: center;
  align-items: center;
}
```

Y con eso ya tienes la foto ocupando todo el fondo de la página.

como se agrega un boton?

En React agregas un botón en el JSX con `<button>`.

Por ejemplo:

```
</> JavaScript
<button>Iniciar sesión</button>
```

Si quieres darle una clase para estilizarlo con CSS:

```
</> JavaScript
<button className="login-button">
  Iniciar sesión
</button>
```

Y en el CSS:

```
</> CSS
.login-button {
  padding: 12px 20px;
  background-color: #0000ff;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
```

Si quieres que haga algo al hacer clic:

```
</> JavaScript
<button onClick={() => alert("Botón presionado")}>
```

pero si el botón está dentro de un formulario de login, normalmente haces esto:

Y la función:

O sea:

crea el botón,

sirve para darle estilo.

Y:

define qué hace.

Para un login, te conviene usar `type="submit"` dentro del `<form>`.

Para un **Register** real, normalmente **sí** necesitas **backend**. El frontend por sí solo no debería conectarse directamente a una base de datos tradicional como PostgreSQL/MySQL, porque tendrías que exponer credenciales y lógica sensible al navegador.

El flujo típico es:

Por ejemplo, en React podrías tener:

Eso manda algo así:

Luego el backend recibe esos datos y hace algo conceptualmente parecido a:

+ Preguntar a ChatGPT Alta

Luego el backend recibe esos datos y hace algo conceptualmente parecido a:

Pero la contraseña **no debería guardarse como texto normal**. El backend debería transformarla con algo como `bcrypt` o `Argon2`.

Sin backend propio, tienes tres alternativas:

- Para una entrega solo de frontend, puedes **simular el registro y guardar usuarios temporalmente en localStorage**.
- Puedes usar una plataforma como **Firebase o Supabase**, donde ellos te dan autenticación y base de datos. Ahí no haces tu propio backend tradicional, pero sigue habiendo un backend; simplemente lo administra el servicio.
- Para una aplicación real con PostgreSQL/MySQL, necesitas un backend o una API intermedia.

Para tu proyecto, si todavía están haciendo solo frontend, podrías hacer algo como:

Eso permitiría "registrar" al usuario en el navegador, pero ojo: no es una base de datos real ni es seguro, especialmente porque estarías guardando la contraseña en el navegador.

La separación que conviene aprender es:

Para una entrega universitaria donde todavía no tienen backend, `localStorage` puede servir para simular el flujo, pero cuando conecten el backend deberían reemplazarlo por un `POST /register`.

pero tengo datos mock, podria meterlos a un json de datos mocks?

Sí, pero con una limitación importante: desde React en el navegador puedes leer un JSON de datos mock, pero no puedes modificar físicamente ese archivo JSON y guardar ahí nuevos usuarios de forma permanente.

Por ejemplo, podrías tener:

Con algo así:

Y en React importarlo:

Después puedes buscar usuarios:

Eso sirve perfecto como mock de login.

El problema aparece con Register. Si haces:

puedes modificar ese arreglo mientras la página está funcionando, pero no estás modificando realmente `users.json`. Si recargas la aplicación, vuelve al JSON original.

puedes monitorizar ese arreglo mientras la pagina esta funcionando, pero no estas modificando realmente users.json. Si recargas la aplicación, vuelve al JSON original.

Entonces, para tu entrega frontend, una solución razonable sería:

```
const {
  users.json
  +
  usuarios mock iniciales
}
Register
+
usuario nuevo
+
localStorage
Login
+
review:
users.json
+
localStorage
```

Por ejemplo, en Register podrías hacer:

```
const handleRegister = (e) => {
  e.preventDefault();

  const newUser = {
    id: Date.now(),
    name: name,
    email: email,
    password: password,
  };

  const registeredUsers =
    JSON.parse(localStorage.getItem("registeredUsers")) || [];

  registeredUsers.push(newUser);

  localStorage.setItem(
    "registeredUsers",
    JSON.stringify(registeredUsers)
  );
};
```

Y después en Login:

```
import mockUsers from "../data/users.json";

const registeredUsers =
  JSON.parse(localStorage.getItem("registeredUsers")) || [];

const allUsers = [...mockUsers, ...registeredUsers];

const userFound = allUsers.find(
  (user) =>
    user.email === email &&
    user.password === password
);
```

```
const allUsers = [...mockUsers, ...registeredUsers];

const userFound = allUsers.find(
  (user) =>
    user.email === email &&
    user.password === password
);
```

Entonces tendrías:

```
const {
  users.json
  +
  usuario mock 1
  +
  usuario mock 2
  +
  usuario mock 3
}
localStorage
+
  usuario registrado 1
  +
  usuario registrado 2
}
+
allUsers
```

Para un proyecto que todavía no tiene backend, me parece una forma bastante lógica de simularlo.

Lo que no haría es intentar escribir directamente sobre users.json desde React, porque el navegador no funciona así. Para modificar el archivo de verdad necesitarías backend, Node, una API, etc.

Y obviamente esto es solo para mocks: guardar contraseñas así en localStorage sería inseguro en una aplicación real.

🔗 📄 📄 📄 📄 📄 📄

ya pero da lo mismo que no queden guardados si recargo la pagina

Sí, entonces puedes hacerlo mucho más simple. Si te da lo mismo perder los usuarios al recargar, ni siquiera necesitas localStorage.

La idea sería cargar tus usuarios mock al iniciar la app y mantenerlos en un useState.

Por ejemplo, tienes:

```
{
  "id": 1,
  "name": "Vicente",
  "email": "vicente@mail.com",
  "password": "1234"
}
```

Y en React haces algo así:

```
const {
  users.json
  +
  usuario mock 1
  +
  usuario mock 2
  +
  usuario mock 3
}
localStorage
+
  usuario registrado 1
  +
  usuario registrado 2
}
+
allUsers
```

```
const {
  users.json
  +
  usuario mock 1
  +
  usuario mock 2
  +
  usuario mock 3
}
localStorage
+
  usuario registrado 1
  +
  usuario registrado 2
}
+
allUsers
```

```
</> JSON
{
  // tus usuarios
}
```

Y en React haces algo así:

```
</> JavaScript
import mockUsers from "../data/users.json";
import { useState } from "react";

function App() {
  const [users, setUsers] = useState(mockUsers);

  return (
    // tus páginas
  );
}
```

Después, en `Register`, para agregar un usuario:

```
</> JavaScript
const newuser = {
  id: users.length + 1,
  name: name,
  email: email,
  password: password
};

setUsers([...users, newuser]);
```

Eso hace:

```
</> Texto sin formato
users iniciales
[
  Vicente,
  Pedro
]

↓ registro de Juan

setUsers(...)

[
  Vicente,
  Pedro,
  Juan
]
```

Y mientras no recargues la página, `Juan` sigue existiendo.

La parte importante es esta: el estado `users` debería estar en un componente superior, por ejemplo `App.jsx`, y no dentro de `Register.jsx`.

Porque si haces esto:

```
</> Texto sin formato
App
├── Login
└── Register

+ Preguntar a ChatGPT Alta 🔊
```

La parte importante es esta: el estado `users` debería estar en un componente superior, por ejemplo `App.jsx`, y no dentro de `Register.jsx`.

Porque si haces esto:

```
</> Texto sin formato
App
├── Login
└── Register
```

puedes pasar los usuarios a ambos:

```
</> JavaScript
<Register users={users} setUsers={setUsers} />
<Login users={users} />
```

Entonces `Register` agrega usuarios:

```
</> JavaScript
function Register({ users, setUsers }) {
```

y `Login` puede buscarlos:

```
</> JavaScript
function Login({ users }) {
```

Por ejemplo:

```
</> JavaScript
const userFound = users.find(
  (user) =>
    user.email === email &&
    user.password === password
);
```

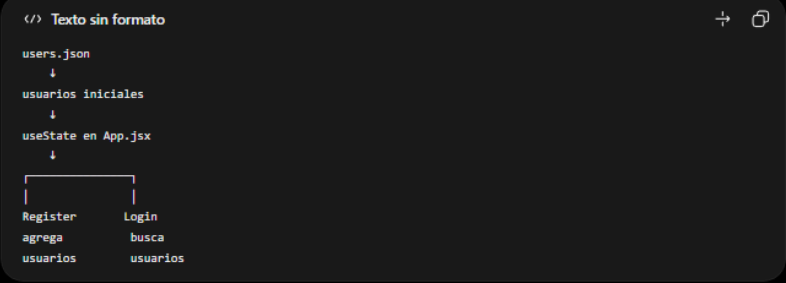
Así tu sistema mock queda:



Y al hacer F5:

```
</> Texto sin formato
+ Preguntar a ChatGPT Alta 🔊
```

Así tu sistema mock queda:



Y al hacer F5:

```
</> Texto sin formato
useState vuelve a cargarse desde users.json
├── ↓
├── los usuarios creados desaparecen
```

Si eso te da lo mismo para la entrega, esta es probablemente la solución más simple y adecuada. No necesitas backend ni `localStorage`.

📄 📤 🔊 🔄 🌐 🗑️ ...

ChatGPT puede cometer errores. Comprueba la información importante.

+ Preguntar a ChatGPT Alta 🔊

Snippet de código entregado para configurar correctamente el eslint:

```
globals: {
  ...builtinModules.reduce((acc, mod) => {
    acc[mod] = "readonly";
    return acc;
  }, {}),
  ...globals.browser,
  ...globals.node
},
},
```

como se usa clamp para darle responsividad a mi página web?

Explicando el uso de clamp() en CSS responsivo.

`clamp()` es una función de CSS que te permite definir un valor que se ajusta de forma fluida entre un mínimo y un máximo, sin necesidad de tantas media queries. Su sintaxis es:

CSS

```
propiedad: clamp(valor-mínimo, valor-preferido, valor-máximo);
```

- **valor-mínimo:** el límite inferior, nunca bajará de ahí.
- **valor-preferido:** normalmente una unidad relativa (como `vw`, `%`, `em`) que hace que el valor cambie según el tamaño de la pantalla.
- **valor-máximo:** el límite superior, nunca subirá de ahí.

Ejemplo con tipografía

CSS

```
h1 {  
  font-size: clamp(1.5rem, 4vw, 3rem);  
}
```

Esto significa: el tamaño de fuente nunca será menor a `1.5rem` ni mayor a `3rem`, pero entre esos límites va a escalar según el ancho de la ventana (`4vw`).

Por qué es útil

- Evita escribir varias media queries solo para ajustar tamaños.
- El cambio es progresivo (fluido), no a saltos como con los breakpoints tradicionales.
- Funciona con cualquier propiedad que acepte un valor numérico: `font-size`, `width`, `margin`, `padding`, `gap`, etc.

Cómo elegir el valor preferido

Una fórmula común para calcular el `vw` ideal es:

$$vw = (\text{valor-máximo} - \text{valor-mínimo}) / (\text{ancho-máximo} - \text{ancho-mínimo}) * 100$$

Por ejemplo, si quieres que el tamaño de fuente pase de 16px (a 320px de ancho) a 24px (a 1200px de ancho):

$$vw = (24 - 16) / (1200 - 320) * 100 \approx 0.909vw$$

Y luego combinas eso con un valor base en `rem`:

CSS

```
font-size: clamp(1rem, 0.5rem + 0.91vw, 1.5rem);
```

</> 

Esa combinación de `rem + vw` es un patrón muy usado porque respeta también la configuración de accesibilidad del usuario (si cambia el tamaño de fuente base del navegador).

Si ya estoy usando clamp, eso es suficiente para darle responsividad a mi pagina web?

Explicando las limitaciones de clamp en diseño responsivo.

No, `clamp()` por sí solo no es suficiente. Es una herramienta muy útil, pero solo resuelve **una parte** del problema de la responsividad: que ciertos valores (tamaños de fuente, anchos, espaciados) se ajusten de forma fluida entre un mínimo y un máximo.

Qué NO cubre `clamp()`

1. Cambios estructurales de layout

`clamp()` no puede reorganizar tu página. Si necesitas que un menú horizontal se convierta en un menú hamburguesa en móvil, o que una grilla de 4 columnas pase a 1 columna, eso lo debes resolver con **media queries**, **Flexbox** o **CSS Grid**.

```
CSS

/* Esto SÍ necesita media query */
@media (max-width: 768px) {
  .menu { display: none; }
  .menu-hamburguesa { display: block; }
}
```

como sería usar media para el navbar ejemplos

Aquí tienes un ejemplo completo de un navbar responsivo usando media queries, con el patrón típico: menú horizontal en escritorio, menú hamburguesa en móvil.

HTML

```
html

<nav class="navbar">
  <div class="navbar-logo">MiLogo</div>

  <button class="navbar-toggle" onclick="document.querySelector('.navbar-links').class
    ≡
  </button>

  <ul class="navbar-links">
    <li><a href="#">Inicio</a></li>
    <li><a href="#">Servicios</a></li>
    <li><a href="#">Nosotros</a></li>
    <li><a href="#">Contacto</a></li>
  </ul>
</nav>
```


CSS base (mobile-first)

CSS

</> 

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 1rem;  
  background: #1e1e2f;  
  color: white;  
}  
  
.navbar-toggle {  
  display: block;  
  background: none;  
  border: none;  
  color: white;  
  font-size: 1.5rem;  
  cursor: pointer;  
}
```

Media query para escritorio

CSS

</> 

```
@media (min-width: 768px) {  
  .navbar-toggle {  
    display: none;  
  }  
  
  .navbar-links {  
    display: flex;  
    flex-direction: row;  
    position: static;  
    width: auto;  
    background: transparent;  
  }  
  
  .navbar-links li {  
    padding: 0 1rem;  
    border-top: none;  
  }  
}
```

para mi juego web, en el login, que codigos de error debo usar en el protocolo json para cada caso?

Read 2 memories >

This info es útil — sé que tu proyecto usa React (frontend) + Node/Koa (backend), con protocolo HTTP + polling. Te dejo una propuesta de códigos de error para el login, pensada para ese stack.

Estructura general de la respuesta de error

```
json
{
  "success": false,
  "error": {
    "code": "AUTH_INVALID_CREDENTIALS",
    "message": "Usuario o contraseña incorrectos"
  }
}
```

Usar un `code` (string constante, en mayúsculas) además del `message` es importante porque el frontend debería decidir qué hacer según el `code` (mostrar cierto texto, redirigir, etc.), mientras que el `message` es solo para mostrar al usuario y puede cambiar sin romper la lógica del cliente.

Códigos sugeridos para login

Código	Caso	Status HTTP sugerido
<code>AUTH_MISSING_FIELDS</code>	Falta usuario o contraseña en el body	400
<code>AUTH_INVALID_FORMAT</code>	Formato inválido (ej. username con caracteres no permitidos)	400
<code>AUTH_USER_NOT_FOUND</code>	El usuario no existe	404 (o 401, ver nota abajo)
<code>AUTH_INVALID_CREDENTIALS</code>	Usuario existe pero contraseña incorrecta	401
<code>AUTH_ACCOUNT_LOCKED</code>	Cuenta bloqueada por intentos fallidos	423
<code>AUTH_ACCOUNT_DISABLED</code>	Cuenta deshabilitada/suspendida	403
<code>AUTH_SESSION_EXPIRED</code>	Token/sesión expirada (si usan JWT o sesiones)	401
<code>AUTH_TOO_MANY_ATTEMPTS</code>	Rate limiting de intentos de login	429
<code>SERVER_ERROR</code>	Error inesperado del servidor	500

es necesario que las peticiones REST tengan un id propio cada una? o un id por jugador?

Designing request ID structure for a game's REST protocol.

Son dos cosas distintas que resuelven problemas diferentes — no es "uno u otro", en general conviene tener **ambos**, pero no todos los endpoints los necesitan igual.

ID de jugador (o de sesión)

Sirve para identificar **quién** hace la petición. Es prácticamente obligatorio en cualquier endpoint autenticado, porque el backend necesita saber a qué jugador/partida corresponde la acción.

La forma recomendada **no** es mandarlo como un campo cualquiera del body, sino en el header de autenticación:

```
GET /api/game/state
Authorization: Bearer <token-de-sesion>
```

como son los mensajes json de un websocket?

Explicando la estructura de los mensajes JSON en WebSocket.

Los mensajes JSON en WebSocket no tienen un formato "oficial" impuesto por el protocolo — WebSocket solo transmite texto (o binario) crudo. El formato JSON que uses es una convención que tú defines. Pero hay patrones muy establecidos que casi todo el mundo sigue. Te muestro la estructura típica.

Estructura básica: "envelope" (sobre)

La convención más común es envolver cada mensaje con un **tipo** y un **contenido**, para que el cliente sepa qué hacer con cada mensaje que llega:

```
json
{
  "tipo": "NOMBRE_DEL_EVENTO",
  "contenido": { ... }
}
```

Esto es exactamente lo que ya vi en tu ejemplo anterior ("tipo": "ESTADO_PARTIDA") — tu equipo ya está usando este patrón, lo cual es buena práctica.

que cosas se pueden resolver por REST y que cosas por WebSocket?
ejemplos de cada uno y los json

La diferencia de fondo

REST (HTTP) funciona como una conversación de pregunta y respuesta: el cliente pide algo, el servidor responde y la conexión termina. El servidor **nunca habla primero**: si nadie le pregunta, no puede avisar nada.

WebSocket abre una conexión que **queda abierta** y por la que ambos lados pueden enviar mensajes cuando quieran. Su gran ventaja es que el servidor **puede avisar sin que le pregunten**.

La pregunta clave para decidir es: ¿el cliente necesita enterarse de algo que él no pidió?

- Si no, porque el cliente siempre inicia y solo espera la respuesta a lo que pidió, conviene **REST**.
- Si sí, porque algo puede pasar en cualquier momento (otro usuario escribe, un precio cambia, el rival juega), conviene **WebSocket**.

Qué conviene resolver por REST

Las operaciones que inicia el propio usuario y que tienen una respuesta concreta:

- Registro, login y logout.
- Consultar datos: un perfil, una lista de productos, un historial.
- Crear, modificar o borrar algo: una publicación, un pedido, una configuración.
- Acciones que deben validarse y responder "funcionó" o "falló, por esta razón".

REST aporta cosas que WebSocket no trae por defecto: **verbos** (**GET**, **POST**, **PUT**, **DELETE**), **códigos de estado** con significado estándar (**200**, **201**, **400**, **401**, **404**, **409**) y URLs que identifican cada recurso.

Ejemplo 1: login

POST /auth/login

```
{
  "email": "ana@correo.com",
  "password": "secreta123"
}
```

Respuesta **200 OK**:

```
{
  "usuario": { "id": 7, "nombre": "Ana" },
  "token": "eyJhbGciOiIi
}
```

Respuesta **401 Unauthorized**:

```
{
  "codigo": "CREDENCIALES_INVALIDAS",
  "mensaje": "El correo o la contraseña no son correctos."
}
```

Ejemplo 2: consultar un recurso

GET /productos/42

Respuesta **200 OK**:

Qué conviene resolver por WebSocket

Todo lo que el servidor tiene que **empujar** hacia el cliente en tiempo real:

- Mensajes de chat nuevos.
- Notificaciones ("alguien comentó tu publicación").
- Presencia: quién está conectado, "está escribiendo...".
- Datos que cambian constantemente: precios, marcadores deportivos, ubicación de un repartidor.
- Juegos multijugador: la jugada del rival, el cambio de turno.
- Edición colaborativa, donde varias personas modifican el mismo documento a la vez.

Como WebSocket no tiene URLs ni verbos, **cada mensaje tiene que decir qué es**. La convención es incluir un campo `tipo`.

Ejemplo 1: llega un mensaje de chat

Servidor → cliente:

```
{
  "tipo": "MENSAJE_NUEVO",
  "contenido": {
    "salaId": "sala-3",
    "autor": { "id": 12, "nombre": "Pedro" },
    "texto": "¿Nos juntamos a las 5?",
    "enviadoEn": "2026-09-20T16:42:10Z"
  }
}
```

como se puede decidir en el protocolo qué información recibe cada jugador y qué se le oculta

El principio base: si no lo puede ver, no se envía

En un juego multijugador, el **servidor** es el único que conoce el estado completo de la partida. Cada jugador recibe solo una **vista**: la parte de ese estado que tiene derecho a conocer.

El error más común es enviarle todo al cliente y ocultar en la interfaz lo que no debería verse. Cualquier jugador puede abrir las herramientas del navegador (pestaña *Network*) y leer el mensaje completo, que es la forma más básica de hacer trampa. Por eso la regla es que **la información oculta no viaja por la red**. Ocultarla solo en la pantalla no sirve.

Paso 1: clasificar cada dato del juego

Haz una lista con todos los datos de la partida y pregúntale a cada uno: **¿quién puede saber esto, y bajo qué condición?** Normalmente caen en cuatro categorías:

Categoría	Quién lo ve	Ejemplos
Público	Todos	Turno actual, puntaje, cartas sobre la mesa, nombre de los jugadores
Privado	Solo su dueño	Tu mano de cartas, tu dinero, tus recursos
Condicional	Depende de una regla del juego	Unidades enemigas dentro de tu rango de visión, cartas que te mostraron
Secreto para todos	Nadie, solo el servidor	Orden del mazo, semilla del azar, próximo evento

Esta tabla debería quedar escrita en el protocolo. Es la especificación de qué se filtra.

Paso 2: un mensaje distinto por jugador

Como cada jugador ve cosas distintas, el servidor **no puede mandar el mismo mensaje a todos**. Por cada cambio, arma una versión para cada uno.

Ejemplo con un juego de cartas tipo póker. Este es el estado completo que conoce el servidor:

```
{
  "mazo": ["K♠", "3♦", "9♣", "..."],
  "mesa": ["A♥", "7♠", "2♦"],
  "jugadores": [
    { "id": 1, "mano": ["Q♥", "Q♠"], "fichas": 500 },
    { "id": 2, "mano": ["J♦", "10♦"], "fichas": 320 }
  ]
}
```

Lo que recibe el Jugador 1:

```
{
  "mesa": ["A♥", "7♠", "2♦"],
  "yo": { "id": 1, "mano": ["Q♥", "Q♠"], "fichas": 500 },
  "rivales": [
    { "id": 2, "cartasEnMano": 2, "fichas": 320 }
  ]
}
```

Lo que recibe el Jugador 2:

```
{
  "mesa": ["A♥", "7♠", "2♦"],
  "yo": { "id": 2, "mano": ["J♦", "10♦"], "fichas": 320 },
  "rivales": [
    { "id": 1, "cartasEnMano": 2, "fichas": 500 }
  ]
}
```

como se puede configurar el navbar para mostrar cosas diferentes en cada vista? no siempre lo mismo? ejemplos

Hay varias formas de hacer esto, dependiendo de qué tan dinámico necesitas que sea. Como tu proyecto usa React, te muestro ejemplos enfocados en ese stack, aunque el concepto general aplica a cualquier framework.

1. Navbar condicional según la ruta (la más común)

Usas el hook de la ruta actual para decidir qué mostrar:

```
jsx
import { useLocation } from 'react-router-dom';

function Navbar() {
  const location = useLocation();

  return (
    <nav className="navbar">
      <div className="navbar-logo">MiJuego</div>

      <ul className="navbar-links">
        <li><a href="/">Inicio</a></li>
```

```

<ul className="navbar-links">
  <li><a href="/">Inicio</a></li>

  {location.pathname === '/' && (
    <li><a href="/como-jugar">Cómo jugar</a></li>
  )}

  {location.pathname.startsWith('/game') && (
    <>
      <li><a href="/game/reglas">Reglas</a></li>
      <li><a href="/game/abandonar">Abandonar partida</a></li>
    </>
  )}
</ul>
</nav>
);
}

```

2. Navbar según estado de autenticación

Muy típico en juegos: mostrar "Iniciar sesión" o el perfil del usuario según si está logueado.

2. Navbar según estado de autenticación

Muy típico en juegos: mostrar "Iniciar sesión" o el perfil del usuario según si está logueado.

```

jsx
function Navbar({ usuario }) {
  return (
    <nav className="navbar">
      <div className="navbar-logo">MiJuego</div>

      <ul className="navbar-links">
        {usuario ? (
          <>
            <li>Bienvenido, {usuario.username}</li>
            <li><a href="/perfil">Perfil</a></li>
            <li><button onClick={cerrarSesion}>Cerrar sesión</button></li>
          </>
        ) : (
          <>
            <li><a href="/login">Iniciar sesión</a></li>
            <li><a href="/registro">Registrarse</a></li>
          </>
        )}
      </ul>
    </nav>
  );
}

```

como estructurarías mi parte de la entrega 2 para desarrollarla desde cero, dime que archivos se deben crear y que contiene cada uno basándose en lo desarrollado en la entrega 1

Acá está el plan completo. Para cada archivo: de dónde sale en la E1, qué contiene y cómo verificarlo antes de seguir.

Orden de construcción

Va de abajo hacia arriba, porque cada pieza necesita la anterior para poder probarse.

1. mocks/flota.js	→ datos, sin dependencias	
2. mocks/eventos.js	→ datos, sin dependencias	
3. BoardCell	→ usa solo CSS	
4. Tablero	→ usa BoardCell + flota.js	
5. PanelRecursos	→ independiente	
6. Preparacion	→ usa Tablero + flota.js	
7. Partida	→ usa Tablero + PanelRecursos + los 3 mocks	
8. App.jsx	→ conecta las rutas	

1. src/mocks/flota.js

Sale de: la tabla de barcos de la E1.

Contiene: un array `tiposBarco` con los 4 tipos, en orden de menor a mayor. Cada objeto lleva:

Campo	Lancha	Fragata	Destructor	Acorazado
<code>tipo</code>	LANCHA	F GATA	DESTRUCTOR	ACORAZADO

Contiene: un array `tiposBarco` con los 4 tipos, en orden de menor a mayor. Cada objeto lleva:

Campo	Lancha	Fragata	Destructor	Acorazado
<code>tipo</code>	LANCHA	FRAGATA	DESTRUCTOR	ACORAZADO
<code>nombre</code>	Lancha	Fragata	Destructor	Acorazado
<code>letra</code>	L	F	D	A
<code>casco</code>	1	2	3	4
<code>movimiento</code>	3	2	2	1
<code>alcance</code>	2	3	4	5
<code>deteccion</code>	3	2	2	1
<code>dado</code>	d4	d6	d8	d10
<code>combustiblePorCasilla</code>	1	1	2	2
<code>municionPorDisparo</code>	1	1	1	2

`nombre` y `letra` no vienen de la E1: son para la interfaz. La letra es lo que se pinta dentro de la casilla.

Exporta además: `statsDe(tipo)`, que busca un tipo en el array. La necesitas porque el estado de partida solo manda el string (`"LANCHA"`), no los stats completos.

Checkpoint: `console.log(statsDe("ACORAZADO").movimiento)` da 1.

2. src/mocks/eventos.is



2. [src/mocks/eventos.js](#)

Sale de: la sección de eventos aleatorios de la E1 y los valores de `eventoRonda.carta` del protocolo (punto 3.1).

Contiene: array de 4 objetos con `carta` y `efecto`:

- `TORMENTA` — Radio de detección -1 para todos los barcos esta ronda
- `BOTIN_FLOTANTE` — Aparece un bono de 2 recursos en una casilla vacía al azar
- `SABOTAJE` — Un jugador elegido al azar pierde 1 de munición
- `MAREA_ALTA` — Las corrientes peligrosas no hacen daño esta ronda

Exporta además: `eventoAlAzar()`.

Ojo con esto: en el juego real la carta la sortea el servidor, nunca el cliente. Acá se hace en el navegador solo porque el backend no existe. Déjalo anotado en un comentario, porque es una violación consciente de una de sus reglas.

3. [src/components/BoardCell/BoardCell.jsx](#) + `.css`

Sale de: los tipos de casilla de la E1 y las reglas de filtrado del protocolo (punto 3.2).

Responsabilidad: pintar una casilla. No sabe qué es un tablero ni qué es una partida.

Props:

x, y	número de la coordenada	
tipo	AGUA_ABIERTA POZO_PETROLERO DEPOSITO_MUNICION NAUFRAGIO CORRIENTE_PELIGROSA CHATARRA null	
barco	{ letra, tipo, casco, aFlote, prop } null	
niebla	boolean	visible

4. [src/components/Tablero/Tablero.jsx](#) + `.css`

Sale de: el mapa 10x10 de la E1 y la estructura del Estado de Partida (punto 3 del protocolo).

Props:

casillas	array de { x, y, tipo }	← de casillasVisibles
flota	array de barcos propios	← de yo.flota
contactos	array de barcos enemigos	← de contactosEnemigos
seleccion	{ x, y } null	
disponibles	array de { x, y }	
zona	{ desde, hasta } null	
onSeleccionar	(x, y) => void null	

Estructura interna:

- `TAMANO = 10`, `LETRAS = ["A"... "J"]`
- `coordenadas` generado con `Array.from({length: 10}, (_, i) => i + 1)` para poder usar `.map()`
- `clave(x, y)` devuelve `"3,5"`, la llave de los índices
- `indexarBarcos(flota, contactos)` junta propios y enemigos en un `Map` por casilla, agregándole a cada uno la `letra` (vía `statsDe`) y el flag `propio`
- El render arma: esquina vacía → 10 letras de columna → por cada fila, un `<Fragment key>` con el número y 10 `<BoardCell>`
- Dentro del doble `.map()`: `niebla = !casilla && !barco`, y `enZona` comparando `y` contra el rango de `zona`

El CSS maneja: `display: grid` con `grid-template-columns: auto repeat(10, 1fr)`, el gap entre casillas, y el estilo de las etiquetas de coordenadas.

Checkpoint: móntalo con `casillas={[]}` y `flota={[]}`. ves ver una grilla de 100 casillas en niebla con

5. `src/components/PanelRecursos/PanelRecursos.jsx` + `.css`

Sale de: los recursos y el sistema de acciones de la E1, y los campos `yo.recursos`, `yo.accionesRestantes`, `ronda`, `eventoRonda` y `rival` del protocolo.

Props:

```
recursos      { combustible, municion, chatarra }
accionesRestantes número
ronda, rondaMaxima números
evento        { carta, efecto } | null
esMiTurno     bool
segundosRestantes número | null
rival         { username, barcosAFlote, conectado } | null
```

Estructura interna:

- `formatearTiempo(segundos)` convierte a `mm:ss` con `padStart`
- Los tres recursos van en un array de objetos y se renderizan con `.map()`
- Las acciones: `Array.from({length: 3}, (_, i) => i < accionesRestantes)` da `[true, true, false]`, que se mapea a las pastillas
- Cuatro bloques: Recursos, Turno, Evento de la ronda, Rival. Los dos últimos solo si llega la prop

Semántica: el contenedor es `<aside>` y cada bloque un `<section>` con `<h2>`. Las listas son `<ul>` / `<li>`.

Nota de diseño: este componente no conoce el estado de la partida. Solo recibe valores y los pinta. Si cambia de dónde salen los recursos, este archivo no se toca.

Checkpoint: móntalo con valores fijos a mano y revisa que las pastillas de acciones reflejen el número que le pasas.

6. `src/pages/Preparacion/Preparacion.jsx` + `.css`

Sale de: la fase de despliegue de la E1 (zona filas 1-4, timeout 2 minutos, colocación automática del servidor) y el endpoint `POST /games/:id/desplegar` (punto 4.7).

Constantes: `ZONA = { desde: 1, hasta: 4 }`, `SEGUNDOS_DESPLIEGUE = 120`.

Estado:

```
colocados      { LANCHA: {x,y}, ... }  objeto indexado por tipo
seleccionado    string del tipo activo
segundos        cuenta regresiva
error           string
```

Funciones fuera del componente (son puras, no dependen del estado de React):

- `libresEnZona(colocados)` — recorre filas 1 a 4 y devuelve las casillas sin barco
- `completarAlAzar(colocados)` — rellena solo los tipos que faltan, eligiendo de las libres. Es lo mismo que hace el servidor al vencer el tiempo
- `armarDespliegue(colocados)` — convierte el objeto al array `barcos` del punto 4.7
- `armarFlota(colocados)` — convierte al formato de `yo.flota`, para poder pintar el tablero
- `nombreCasilla({x,y})` — devuelve `"C3"`

Handlers:

- `handleCasilla(x, y)` — valida que `y` esté en la zona; si la casilla ya tiene barco lo selecciona en vez de apilar; si no, lo coloca y salta al siguiente tipo sin colocar
- `handleAlAzar`, `handleLimpiar`
- `handleConfirmar` — exige los 4 y navega a `/tablero` pasando la flota por `state`

7. `src/pages/Partida/Partida.jsx` + `.css`

Sale de: el sistema de turnos de la E1, y los puntos 4.8, 4.9 y 4.10 del protocolo.

Constantes: `TAMANO = 10`, `ACCIONES_POR_TURNO = 3`, `ACCIONES_POR_BARCO = 2`, `SEGUNDOS_TURNO = 180`.

Estado:

<code>estado</code>	el objeto Estado de Partida completo
<code>seleccionado</code>	<code>barcoId</code> null
<code>segundos</code>	cuenta regresiva del turno
<code>mensaje</code>	feedback de la última acción

Funciones fuera del componente:

- `copiar(objeto)` — clon profundo con `JSON.parse(JSON.stringify())`, para no mutar el mock
- `distancia(origen, destino)` — Manhattan
- `armarEstadoInicial(flotaDesplegada)` — si viene flota desde preparación arranca en ronda 1 con recursos iniciales (12/8/0); si no, usa el mock tal cual
- `destinosPosibles(barco, estado)` — filtra las 100 casillas por tres condiciones: dentro del movimiento, sin barco encima, y que alcance el combustible

Valores derivados en cada render (no son estado, se calculan):

<code>enCurso</code>	<code>estado.estado === "EN_CURSO"</code>
<code>esMiTurno</code>	<code>jugadorActivo === yo.jugadorId</code>
<code>barcoSeleccionado</code>	busca en la flota por id, y que esté a flote
<code>usadasPorBarco</code>	<code>accionesPorBarco[id] 0</code>
<code>puedeMover</code>	las cuatro condiciones juntas
<code>destinos</code>	<code>[]</code> si no puede mover

`terminarTurno()` — el corazón de tu parte:

- Calcula a quién le toca: alterna entre `yo.jugadorId` y `rival.jugadorId`
- `meToca` = el turno volvió a ti
- La ronda sube **solo si** `meToca`
- Si la ronda supera `rondaMaxima`, reemplaza el estado por `partida.terminada`
- Carta nueva **solo si** `meToca`
- Acciones repuestas **solo si** `meToca`
- Limpia la selección y reinicia el reloj a 180

`handleCasilla(x, y)` valida en el mismo orden que el servidor (punto 6 del protocolo), y si algo falla **no descuenta nada**:

partida activa → es tu turno → ¿clickeaste un barco propio? entonces seleccionar
→ ¿hay barco seleccionado? → ¿quedan acciones? → ¿ese barco usó sus 2?
→ ¿la casilla está en destinos? → mover

`mover(destino)` calcula `pasos × combustiblePorCasilla`, descuenta combustible, resta una acción, suma al contador de ese barco y actualiza su casilla con `.map()`.

Render: header, tablero con el mensaje debajo, botones de turno y abandonar, `PanelRecursos` al costado, y el bloque de resultado final si `resultadoFinal` no es null.

El botón de turno es dual: "Terminar turno" si es tuyo, "Simular turno del rival" si no. Sin backend no hay rival que juegue.

Checkpoint: mueve un barco y verifica que bajen combustible y acciones; termina turno y comprueba que el panel diga "Turno del rival"; simula el del rival y verifica ronda +1, acciones en 3 y carta nueva.

8. `src/App.jsx` (modificar)

Dos imports y dos rutas:

```
<Route path="/preparacion" element={<Preparacion />} />
<Route path="/tablero" element={<Partida />} />
```

Y en `Menu.jsx`, que el botón de entrar lleve a `/preparacion` y no directo al tablero — porque según el protocolo la partida pasa por el estado `DESPLIEGUE` antes de `EN_CURSO`.

Dos reglas transversales

Ninguna decisión de juego se toma en el cliente. El movimiento y el cambio de turno están en el navegador solo porque no hay servidor. Cuando llegue la E3, esa lógica se reemplaza por llamadas a `POST /games/:id/acciones` y `POST /games/:id/terminar-turno`, y el cliente vuelve a ser solo pintura. Escribe el código pensando en eso: mientras más aislada esté la lógica, menos duele después.

Nada de estilos inline y nada de frameworks CSS. Un `.css` por componente, `box-sizing: border-box`, Grid para el tablero, Flexbox para lo demás.

Si quieres esto como archivo en `docs/` o en tu escritorio para tenerlo al lado mientras programas, dime dónde y te lo dejo.

1. ¿Por qué la celda es un `<button>` y no un `<div>`?

Porque una casilla del tablero **es un control que se acciona**, no un contenedor. Y al usar `<button>` el navegador te regala cuatro cosas que con un `<div>` tendrías que escribir a mano.

1. Foco por teclado. Un `<button>` entra en el orden de tabulación solo. Un `<div>` no: necesitarías `tabIndex="0"` en las 100 celdas.

2. Activación con Enter y Espacio. El botón dispara su `onClick` con el teclado sin que hagas nada. El `div` solo responde al mouse; tendrías que agregar un `onKeyDown` y filtrar las teclas.

3. El atributo `disabled`. Esta es la que más te sirve en tu código. En `BoardCell.jsx` tienes:

```
disabled={!onClick}
```

Esa línea sola hace tres cosas: bloquea el click, saca la celda del orden de tabulación, y hace que un lector de pantalla la anuncie como deshabilitada. Es lo que permite que el mismo componente funcione como tablero interactivo y como tablero de solo lectura. Con un `<div>` no existe `disabled`: tendrías que manejar los tres comportamientos por separado.

4. El rol para lectores de pantalla. Se anuncia como "botón", así que el `aria-label` que armas en `describir()` se lee como una opción accionable. En un `<div>` ese mismo texto se leería como texto suelto, sin indicar que se puede clicar.

La comparación concreta

Esto es lo que tendrías que escribir para igualar el comportamiento con un `div`:

```
<div
  role="button"
```

La comparación concreta

Esto es lo que tendrías que escribir para igualar el comportamiento con un div:

```
<div
  role="button"
  tabIndex={onClick ? 0 : -1}
  aria-disabled={!onClick}
  onClick={manejarClick}
  onKeyDown={(e) => {
    if (e.key === "Enter" || e.key === " ") {
      e.preventDefault();
      manejarClick();
    }
  }}
/>
```

Cinco atributos y un handler, repetidos en 100 celdas, para llegar a lo que `<button type="button" disabled={!onClick}>` ya hace.

Y hay una razón del enunciado

La Entrega 2 dice literalmente que el HTML semántico es obligatorio y que no se debe estructurar el sitio exclusivamente con `<div>`. Un tablero de 100 divs clickeables es justo el caso que penalizan.

Lo que cuesta

No es gratis: el botón trae estilos por defecto del navegador que hay que resetear. Por eso en `BoardCell.css` la clase `.celda` define `padding: 0`, su propio `background-color`, su propio `border` y su propia `font-family` — si no, saldría con el aspecto gris de un botón de formulario.

En una frase, para decirlo tú: la celda se clickea, así que es un botón; usar `<div>` obligaría a reimplementar a mano el foco, el teclado, el estado deshabilitado y el rol de accesibilidad que el `<button>` ya trae.

¿Por qué el tablero usa `<Fragment>` y no `<div>` para cada fila?

Porque **CSS Grid solo posiciona a los hijos directos** del contenedor. Cualquier elemento que metas en medio rompe la grilla.

Qué pasa con un div

`.tablero` está declarado así en `Tablero.css`:

```
display: grid;
grid-template-columns: auto repeat(10, 1fr);
```

Son 11 columnas: una angosta para el número de fila, y diez iguales para las casillas. Para que eso funcione, cada fila tiene que emitir **11 elementos hermanos directos** dentro del grid.

Si envolvieras cada fila en un `<div>`, ese div pasaría a ser un único ítem del grid. Ocuparía **una sola celda** — la primera columna de la primera fila — y los 11 elementos de adentro se acomodarían como contenido normal dentro de ese espacio. El resultado es un tablero colapsado: 10 divs apilados en la columna izquierda, con las casillas amontonadas dentro de cada uno.

El grid no "ve" a los nietos. Solo a los hijos.

Por qué Fragment y no nada

Acá está el detalle fino. El `.map()` tiene que devolver **un solo elemento** por iteración, y React exige una `key` en cada ítem de una lista. Entonces necesitas un envoltorio que agrupe sin generar un nodo en el DOM.

El Fragment hace exactamente eso: agrupa en el JSX y desaparece al renderizar. Los 11 elementos quedan como hijos directos del grid.

Son 11 columnas: una angosta para el número de fila, y diez iguales para las casillas. Para que eso funcione, cada fila tiene que emitir **11 elementos hermanos directos** dentro del grid.

Si envolvieras cada fila en un `<div>`, ese div pasaría a ser un único ítem del grid. Ocuparía **una sola celda** — la primera columna de la primera fila — y los 11 elementos de adentro se acomodarían como contenido normal dentro de ese espacio. El resultado es un tablero colapsado: 10 divs apilados en la columna izquierda, con las casillas amontonadas dentro de cada uno.

El grid no "ve" a los nietos. Solo a los hijos.

Por qué Fragment y no nada

Acá está el detalle fino. El `.map()` tiene que devolver **un solo elemento** por iteración, y React exige una `key` en cada ítem de una lista. Entonces necesitas un envoltorio que agrupe sin generar un nodo en el DOM.

El Fragment hace exactamente eso: agrupa en el JSX y desaparece al renderizar. Los 11 elementos quedan como hijos directos del grid.

Pero hay una restricción: **la forma corta `<...>` no acepta `key`**. Por eso en `Tablero.jsx` está el import explícito arriba:

```
import { Fragment } from "react";
```

y después:

```
{coordenadas.map((y) => (  
  <Fragment key={`fila-${y}`}>  
    <span className="tablero-label">{y}</span>  
    {coordenadas.map((x) => <BoardCell ... />)}  
  </Fragment>  
))}
```

Si usaras `<...>`, React te tiraría un warning de key faltante en cada fila. Ese es el único motivo por el que ahí aparece `Fragment` con nombre completo y no el atajo.

Por qué está diseñado así

Sale de una regla que ustedes escribieron en la E1. En [docs/protocolo_json.md](#), punto 3.2:

`casillasVisibles` — Solo las que están dentro del radio de detección de algún barco propio

El servidor arma **un mensaje distinto por jugador** y filtra antes de enviar. Lo que no puedes ver no viaja.

La razón es de seguridad, no de comodidad. Si existiera un campo `oculta: true`, el dato seguiría estando en el JSON: bastaría abrir la pestaña Network del navegador para leer la posición completa de la flota enemiga. El juego entero se basa en información oculta, así que esconderla con CSS sería decorativo. La única forma real de ocultar algo es no mandarlo.

Por eso el cliente no puede "descubrir" nada por su cuenta: solo pinta lo que le llegó.

Los cuatro niveles de conocimiento

Fíjate que la condición es `!casilla && !barco`, con las dos negaciones. No es casualidad: una celda puede tener barco sin tener terreno conocido, porque los barcos vienen en arrays distintos de las casillas. De ahí salen cuatro estados:

Situación	Qué significa
No llegó nada	Niebla. No sabes qué hay
Llegó la casilla, sin barco	Ves el terreno: agua, pozo, naufragio
Llegó un barco con <code>visible: true</code>	Lo estás viendo ahora, y su <code>casco</code> es el real
Llegó un barco con <code>visible: false</code>	Contacto perdido: su última posición conocida, no dónde está ahora

La alternativa que existe

Se podría usar un `<div>` con `display: contents`, que le dice al navegador que ignore la caja del div y deje que sus hijos participen del grid del abuelo. Funciona, pero tiene un problema conocido: varios navegadores sacan el elemento del árbol de accesibilidad al aplicarlo, y se pierden roles y etiquetas. El Fragment logra lo mismo sin ese riesgo, porque el nodo directamente nunca existe.

La otra opción sería un solo `.map()` sobre 110 elementos calculando fila y columna con divisiones y módulo. Funciona igual, pero se lee bastante peor.

En una frase: el grid solo acomoda a sus hijos directos, así que las filas no pueden tener un contenedor propio; el Fragment agrupa los 11 elementos en el JSX sin crear un nodo en el DOM, y se importa con nombre porque la versión corta `<>` no admite `key`.

¿De dónde sale la niebla de guerra, si no hay ningún campo que diga "oculta"?

De la **ausencia**. La niebla no es un dato, es lo que queda cuando no llega ningún dato.

En `Tablero.jsx`, dentro del doble `.map()`:

```
const casilla = mapaCasillas.get(clave(x, y));
const barco = mapaBarcos.get(clave(x, y));
const niebla = !casilla && !barco;
```

Si para esa coordenada no vino ni una casilla en `casillasVisibles` ni un barco en `flota` / `contactosEnemigos`, entonces el jugador no sabe nada de ese punto del mapa. Eso es la niebla.

Por qué está diseñado así

El cuarto caso es el que se pinta con el círculo rojo punteado. No es niebla, es memoria desactualizada. Y hay un quinto matiz: un enemigo dentro del array pero fuera de tu radio llega con `casco: null`, o sea lo ubicas pero no sabes cuánto daño aguanta.

El detalle que anticipa la E3

En el modelo de datos de ustedes hay una tabla `casilla_revelada`, que guarda qué ha descubierto cada jugador. Existe porque cada uno tiene su propia versión del mapa, y sin ella la niebla se reiniciaría al recargar la página. Hoy el mock no persiste nada, pero el modelo ya lo tiene previsto para cuando exista el backend.

En una frase: la niebla no se marca, se deduce — el servidor solo manda lo que el jugador puede ver, así que toda coordenada sobre la que no llegó información se pinta como inexplorada, y eso es lo único que impide que alguien lea la flota enemiga desde la consola del navegador.

1. ¿Por qué `casco: null` y `casco: 0` son cosas distintas?

¿Por qué la ronda no sube cada vez que alguien termina su turno?

`casco: null` vs `casco: 0`

Son dos cosas de naturaleza distinta: `null` es falta de información, `0` es información.

- `casco: null` → "no sé cuánto casco le queda". El barco enemigo está fuera del radio de detección de tu flota. Sabes dónde está, pero no en qué estado.
- `casco: 0` → "sé que está hundido". Es un valor medido, no una ausencia.

Salen de dos reglas de tu propio protocolo, punto 3.2:

`contactosEnemigos[].casco` — El valor real si el barco está dentro del radio, `null` si no
`aFlote` — cuando se hunde, no se borra del array, queda con `aFlote: false`, `casco: 0` y `visible: true` en su casilla final

El barco hundido **no desaparece** del mensaje, justamente para que puedas pintar el marcador de hundido en su última casilla.

En el código, `BoardCell.jsx`:

```
{barco && barco.aFlote && barco.casco !== null && (  
  <span className="celda-casco">{barco.casco}</span>  
)}
```

Ese `!== null` explícito es a propósito. En JavaScript, `0` es *false*: si hubieras escrito `barco.casco &&`, un barco con casco 0 se comportaría igual que uno con casco desconocido y no podrías distinguirlos. Es la trampa clásica del lenguaje, y acá importa de verdad.

Por qué importa en el juego: si pintaras el `null` como 0, el jugador vería que todos los enemigos que no alcanza a ver están a un disparo de hundirse. Es exactamente la lectura táctica contraria a la realidad.

La misma semántica aparece al disparar (punto 4.8): si le pegas a un barco que sigue a flote y está fuera de tu radio, el servidor responde `cascoRestante: null` — te confirma el impacto, pero no te regala la información de cuánto le queda.

..

Por qué la ronda no sube cada turno

Porque una ronda son los dos turnos, no uno.

```
const meToca = siguiente === anterior.yo.jugadorId;  
const ronda = meToca ? anterior.ronda + 1 : anterior.ronda;
```

La ronda avanza solo cuando el turno vuelve al jugador que parte. Si lo hicieras en cada `terminarTurno`, se romperían tres cosas:

- 1. El límite de partida.** La regla dice que la partida termina a las 20 rondas, que son 40 turnos. Subiendo la ronda en cada turno, el juego se acabaría a la mitad: 20 turnos, 10 por jugador.
- 2. La carta de evento.** La regla de la E1 dice que al inicio de cada ronda el servidor revela una carta *que afecta a ambos por igual*. Si la ronda avanzara por turno, cada jugador jugaría bajo una carta distinta y se perdería la simetría. Por eso en el código la carta se renueva con la misma condición:

```
eventoRonda: meToca ? eventoAlAzar() : anterior.eventoRonda
```

Si te toca una Tormenta, te afecta a ti y al rival durante los dos turnos. Ese es el punto del evento.

- 3. Las acciones.** Se reponen con la misma condición:

```
yo: meToca  
? { ...anterior.yo, accionesRestantes: ACCIONES_POR_TURN0, accionesPorBarco: {} }  
: anterior.yo
```

Las 3 acciones vuelven cuando **te toca jugar**, no cuando **pasas** el turno. Si las repusieras al pasar, el panel mostraría "3 acciones disponibles" durante *cada* el turno del rival, que es información falsa.

Las 3 acciones vuelven cuando **te toca jugar**, no cuando pasas el turno. Si las repusieras al pasar, el panel mostraría "3 acciones disponibles" durante todo el turno del rival, que es información falsa.

Un contraste útil: no todo es por ronda. La regla de recolección de recursos se evalúa **al comenzar el turno**, no al inicio de la ronda — si ocupas un pozo petrolero, cobras tus +2 de combustible cada vez que te toca. Ronda y turno son dos relojes distintos, y hay reglas colgando de cada uno.

En una frase: la ronda agrupa un turno de cada jugador, así que solo avanza cuando el ciclo se cierra; de ahí cuelgan el límite de 20, la carta de evento compartida y la reposición de acciones.

¿Por qué los barcos colocados se guardan en un objeto y no en un array?

Porque la regla del juego es "uno de cada tipo", y un objeto indexado por tipo **hace que esa regla se cumpla sola**.

```
const [colocados, setColocados] = useState({});  
// queda como: { LANCHAS: {x:1,y:3}, FRAGATA: {x:3,y:4}, ... }
```

Lo que te sale gratis

1. No puedes duplicar un barco. El punto 4.7 del protocolo exige exactamente cuatro barcos, uno de cada tipo. Con un objeto, colocar dos lanchas es imposible: la segunda sobrescribe a la primera, porque comparten clave. Con un array tendrías que validar duplicados a mano, y acordarte de hacerlo en cada punto donde se agrega.

2. Colocar y mover son la misma operación.

2. Colocar y mover son la misma operación.

```
setColocados({ ...colocados, [seleccionado]: { x, y } });
```

Esa línea sirve tanto para poner un barco por primera vez como para moverlo a otra casilla. Con un array serían dos caminos distintos: un `push` si no está, o un `findIndex` más reemplazo si ya estaba.

3. Preguntar por un tipo es directo. Al renderizar la lista de la flota necesitas saber dónde está cada barco:

```
{colocados[barco.tipo] ? nombreCasilla(colocados[barco.tipo]) : "-"};
```

Con un array sería un `.find()` por cada fila, en cada render.

4. "Cuáles faltan" es una negación simple. `!colocados[barco.tipo]`. De ahí salen dos comportamientos: el salto automático al siguiente barco sin colocar, y el relleno de `completarAlAzar`, que solo toca los que faltan.

El detalle que vale la pena entender

El orden de la flota no lo guarda el objeto: lo guarda `tiposBarco` en [mocks/flota.js](#). El render itera esa lista y consulta el objeto:

```
{tiposBarco.map((barco) => ( ... colocados[barco.tipo] ... ))}
```

Un array de colocados llevaría información de orden que es redundante, y que podría quedar desincronizada del orden canónico de la flota.

Y la conversión pasa en el borde

Esto es lo más útil de la decisión. El protocolo **sí quiere un array**:

Y la conversión pasa en el borde

Esto es lo más útil de la decisión. El protocolo **sí quiere un array**:

```
{ "barcos": [ { "tipo": "LANCHA", "casilla": { "x": 2, "y": 1 }, ... } ] }
```

Entonces el objeto es la forma interna de trabajo, y se convierte a array recién al armar el request:

```
function armarDespliegue(colocados) {  
  return tiposBarco.map((barco) => ({  
    tipo: barco.tipo,  
    casilla: colocados[barco.tipo]}));  
}
```

El patrón general: **la forma interna se elige por las operaciones que haces, la forma externa la dicta el contrato**. No tienen por qué coincidir, y forzarlas a coincidir suele ser peor.

Cuándo sería al revés

Para ser justos: si el juego permitiera varios barcos del mismo tipo — como el Battleship original, con dos destructores — el objeto no serviría y el array sería lo correcto. La decisión depende de la regla "uno de cada tipo", que es de ustedes. Si algún día cambian esa regla de balance, esta estructura es lo primero que hay que rehacer.

En una frase: se guarda por tipo porque la regla dice uno de cada tipo, y así la estructura impide el duplicado en vez de tener que validarlo; el array del protocolo se construye recién al enviar.